AE410 Lab 3: Parallel Computing and Comparison to Experimental Values

Lab Session: Monday, February 23, 2026
Due: Monday, March 2, 2026, 9am CST

Nikita Kovalov
Junior in Aerospace Engineering at the University of Illinois in Urbana-Champaign

This laboratory assignment will first, conduct a parallel computing time study comparing the convergence time of a 250-iteration process with different number of processes. The simulation we are conducting this experiment on is an airfoil at various dangles of attack. Afterwards, we will perform a validation of our solution by comparing our data to experimental data as well as other public solver data.

I. Parallel Computing Time Study

For parallel computing, SU2 implements Message Passing Interface (MPI). MPI is a way to exchange data between different processes on a CPU so that they can work together in parallel. For the processes to talk to each other, they can either share memory or send information over network, these are called shared memory models and distributed memory models respectively [1]. When parallel computing is enabled, a CFD software will usually divide the computational mesh into smaller subdomains, with each processor in charge of solving the governing equations within it's assigned region. This process allows many portions of the flow to be computed simultananeosly, reducing simulation time. However, communication between processors through MPI introduce overhead, leading to a reducing the decrease in computation time per added processor.

A parallel computing time study was conducted using PowerShell's measure command to time the simulation with various number of processors. The simulation used is a NACA 0012 airfoil at $\alpha = 10°$ which ran for 250 iterations. The simulation was run on my personal laptop with the hardware specifications listed in Table 1. Since the CPU provides 22 logical processors, the simulation was performed using processor utilization counts of $n = \{1, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22\}$.

Table 1 Hardware specifications.

Component	Specification
Processor	Intel Core Ultra 7 155H
Base Clock Speed	3.8 GHz
Boost Clock Speed	4.8 GHz
Physical Cores	16
Logical Processors (Threads)	22
Core Configuration	6 Performance + 8 Efficiency + 2 Low-Power Efficiency
System Type	Laptop

A. Convergence time plot

Figure 1 shows the execution time required to complete 250 solver iterations as a function of the number of MPI processes used in the simulation.

B. Discussion

Increasing the number of logical processes significantly decreased the execution time of the simulation. As additional processes were added, the computational workload was distributed across more processors, allowing the CFD solver to complete iterations more quickly.

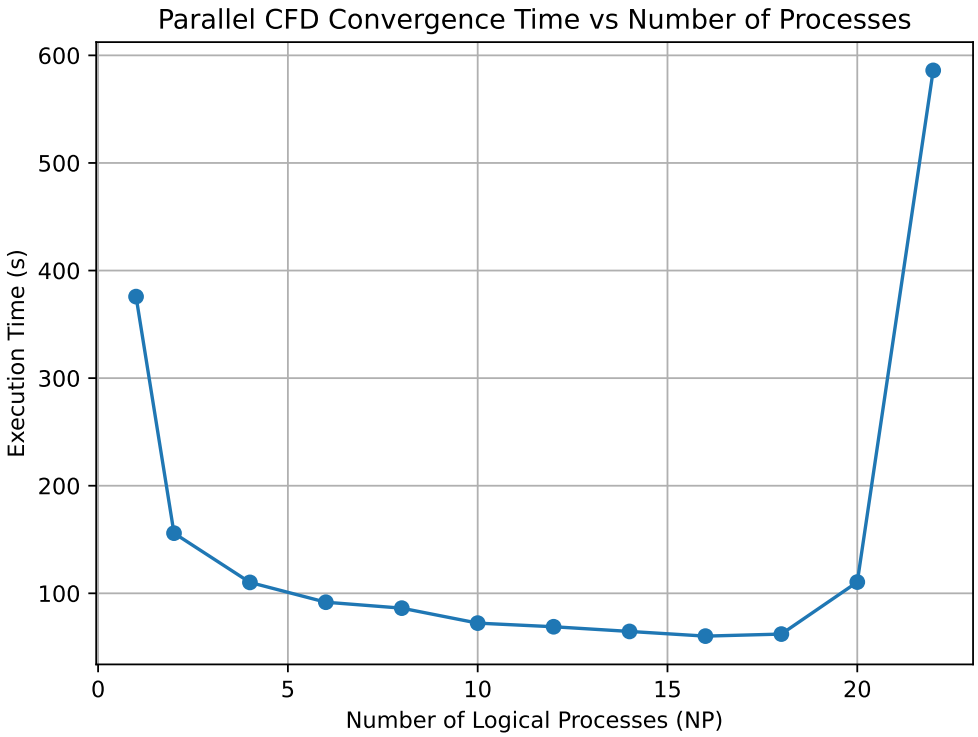

Fig. 1 Execution time required to complete 250 solver iterations as a function of the number of MPI processes.

However, the improvement in execution time diminished as more processes were added. This behavior is likely due to the increased communication cost between MPI processes, as each processor must exchange boundary data with neighboring subdomains during each iteration. As the number of processes increases, the overhead associated with this communication becomes more significant relative to the computational work performed by each processor.

The execution time began to increase once the number of processes exceeded $n = 16$. This is likely due to hardware limitations and operating system as well as background task overhead on the laptop. This matches closely with the rule of thumb of allocating around 2/3rds of your processors to SU2 that was discussed during the tutorial.

Interestingly, the execution time increased over five-fold going from 20 processors to 22. This is is most likely due to oversubscription, where multiple processes compete for the same CPU resources and are repeatedly scheduled by the operating system. This is because although the laptop indicated 22 logical processors, system monitoring showed that only about 20 threads were actively performing computations while running the simulation. The two idle processors were probably low-power efficiency (LPE) cores, which upon further research is only used for by the system for video playback. As a result, when $n = 22$ processes were launched, more MPI processes were created than there were effective compute threads available, resulting in context switching and scheduling overhead which significantly increased the overall execution time.

II. Comparison of results to experimental data

A. Lift and drag comparison

Figures 2 and 3 compare the SU2 CFD predictions against the experimental data of Ladson et al., along with the additional CFD results reported in Table 1. The lift coefficient generally increases with angle of attack, and the SU2 results follow the same overall trend as the experimental and reference CFD data. The drag polar also shows the expected increase in drag with increasing lift, and the SU2 predictions appear to remain within the overall spread of the comparison data.

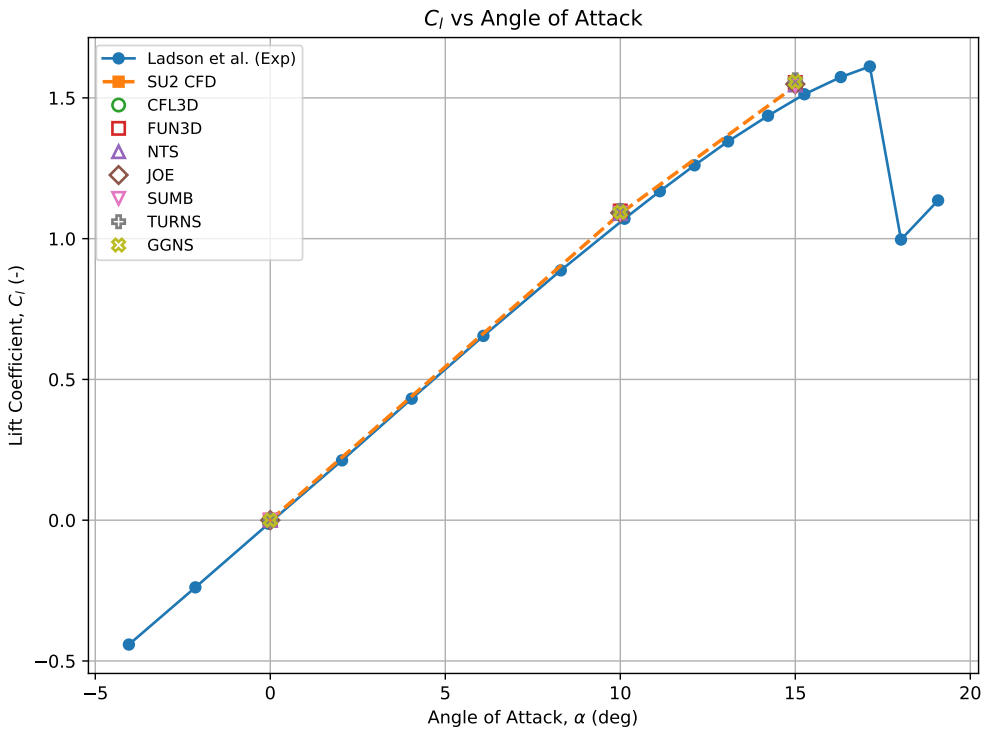

Fig. 2 Comparison of lift coefficient as a function of angle of attack for the SU2 results, experimental data from Ladson et al., and reference CFD results from Table 1.

B. Pressure coefficient distribution at $\alpha = 10°$

Figure 4 compares the SU2 pressure coefficient distribution at $\alpha = 10°$ with the experimental measurements of Gregory and O'Reilly. The CFD prediction captures the general shape of the pressure distribution, including the strong suction peak near the leading edge and the gradual recovery in pressure toward the trailing edge. Overall, the agreement indicates that the solver is able to reproduce the main aerodynamic loading trends on the airfoil at this operating condition.

C. Discussion of CFD validity

Based on these comparisons, the current CFD workflow appears to be sufficiently valid for predicting the overall aerodynamic behavior of the NACA 0012 airfoil at low to moderate angles of attack, particularly for integrated quantities such as lift and drag and for the general shape of the surface pressure distribution. The approach appears most reliable in attached or mostly attached flow regions, where the assumptions of the solver are more appropriate. In this case, the validity of the workflow depends on assumptions including a fixed airfoil geometry, the selected turbulence modeling approach, the numerical discretization used by SU2, and the ability of a steady RANS-based solution to adequately represent the experimental flow conditions.

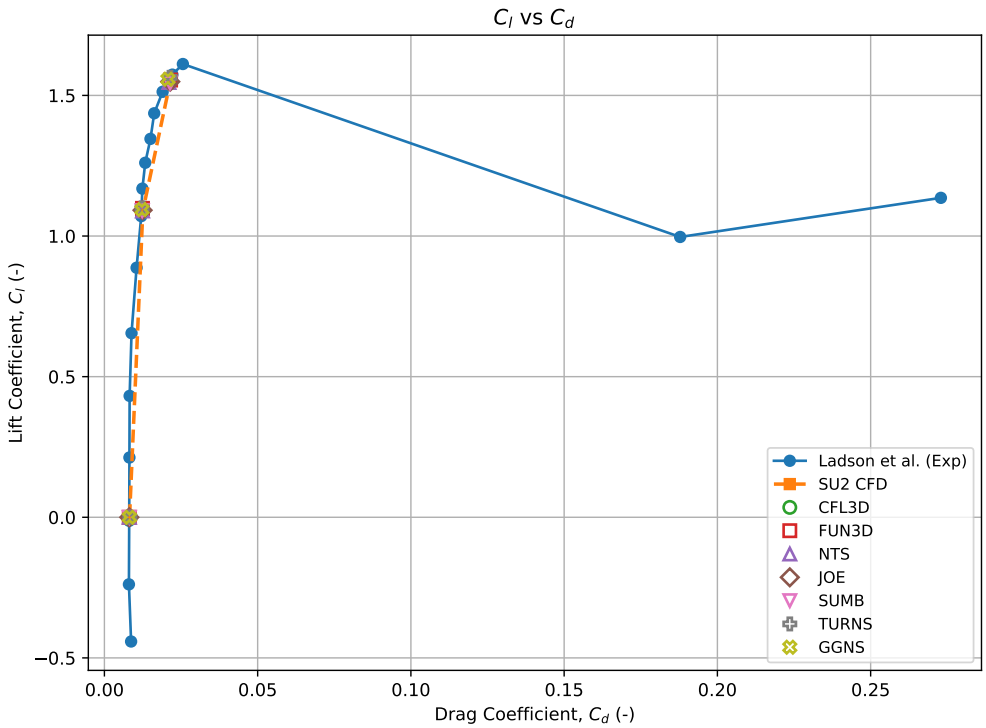

Fig. 3 Comparison of lift coefficient as a function of drag coefficient for the SU2 results, experimental data from Ladson et al., and reference CFD results from Table 1.

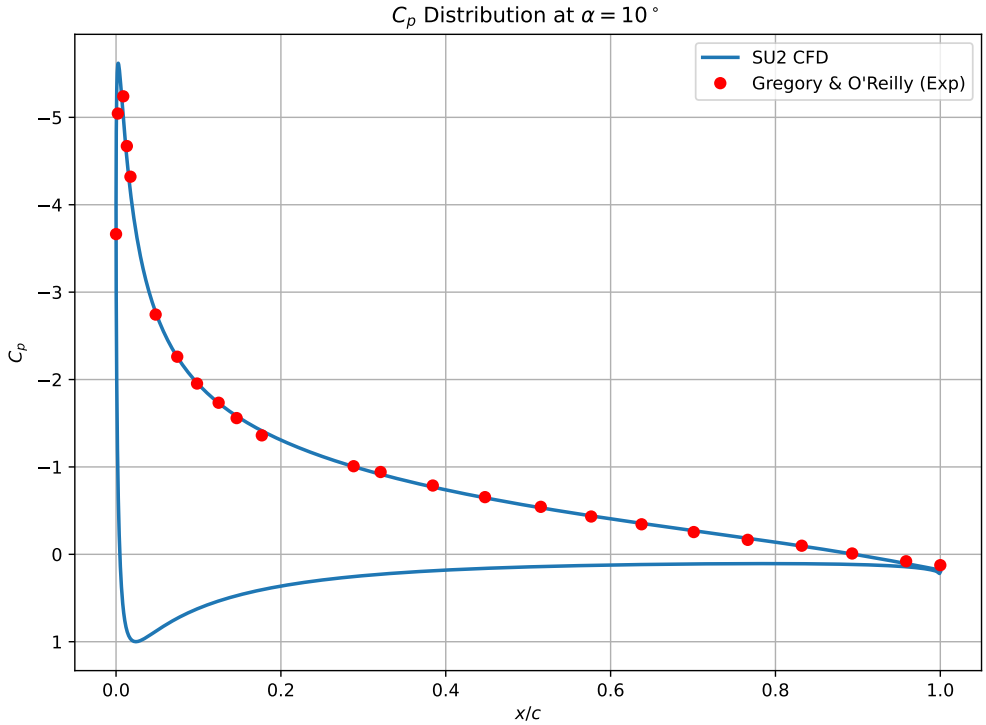

Fig. 4 Comparison of pressure coefficient distribution at $\alpha = 10^\circ$ between the SU2 CFD results and the experimental data of Gregory and O'Reilly.

Appendix

The github repo including the gmsh, Su2, and Paraview files as well as the python code used for post-processing, data analysis, and figure generation are available at: `https://github.com/nikitaaa27/AE462-git/tree/main/Lab3`

References

[1] CU Research Computing, "Fundamentals of Parallel Programming," `https://curc.readthedocs.io/en/latest/programming/parallel-programming-fundamentals.html`, 2024. Accessed: 2026-03-11.